

DOA

Handin

2

Date: 21-09-2026

#1

Stud.nr.: 202507614

Navn: Moein Khaksar

#2

Stud.nr.: 202411378

Navn: Robert Øster Klastrup

Indhold

Exercise 1 - List abstract datatype	3
Exercise 2 - Implementing a stack with an array	6
Exercise 3 – Implementation of Queue.....	8
Exercise 5 – Implement a set.....	12
Exercise 6 - implement map using STL vector	15
Exercise 7 - Quadratic probing	18

Exercise 1 - List abstract datatype

a)

```
exercise1 > C linked_list.h ...
1 #pragma once
2 #include "list.h"
3 #include <iostream>
4
5 template <typename Object>
6 class Linkedlist : public List<Object>{
7 private:
8     struct Node{
9         Object data;
10        Node *next;
11    };
12
13    int theSize;
14    Node *head;
15    Node *tail;
16
17    Node* find_second_last_node(){
18        if (head == nullptr)
19            return nullptr;
20        Node *last = head;
21        while (last->next != tail) {
22            last = last->next;
23        }
24        return last;
25    }
26
27 public:
28     ~Linkedlist(){
29         clear();
30         delete head;
31         delete tail;
32     }
33
34     Linkedlist(){
35         theSize = 0;
36         head = new Node;
37         tail = new Node;
38         head->next = tail;
39         tail->next = nullptr;
40     }
41
42     int size(){
43         return theSize;
44     }
45     bool empty(){
46         return head->next == tail;
47     };
48 }
```

```
exercise1 > C linked_list.h ...
1 #pragma once
2 #include "list.h"
3 #include <iostream>
4
5 template <typename Object>
6 class Linkedlist : public List<Object>{
7 private:
8     struct Node{
9         Object data;
10        Node *next;
11    };
12
13    int theSize;
14    Node *head;
15    Node *tail;
16
17    Node* find_second_last_node(){
18        if (head == nullptr)
19            return nullptr;
20        Node *last = head;
21        while (last->next != tail) {
22            last = last->next;
23        }
24        return last;
25    }
26
27 public:
28     ~Linkedlist(){
29         clear();
30         delete head;
31         delete tail;
32     }
33
34     Linkedlist(){
35         theSize = 0;
36         head = new Node;
37         tail = new Node;
38         head->next = tail;
39         tail->next = nullptr;
40     }
41
42     int size(){
43         return theSize;
44     }
45     bool empty(){
46         return head->next == tail;
47     };
48 }
```

The only thing to note was for push/pop middle, we made it so, that the position couldn't be at the beginning nor the end of the linked list. Which would then remove the need for the pop/push back/front. Note the list starts at position 0

b)

```

int main() {
    LinkedList<int> lst;

    std::cout << "Test empty list\n";
    std::cout << "Empty? " << (lst.empty() ? "yes" : "no") << "\n";
    std::cout << "Size: " << lst.size() << "\n";

    std::cout << "\nTest push_front\n";
    lst.push_front(20);
    lst.push_front(10);
    lst.print(); // expected: h-> [10]->[20]->tail

    std::cout << "\nTest push_back\n";
    lst.push_back(30);
    lst.push_back(40);
    lst.print(); // expected: h-> [10]->[20]->[30]->[40]->tail

    std::cout << "\nTest push_middle\n";
    lst.push_middle(2, 99); // insert at position 2
    lst.print(); // expected: h-> [10]->[20]->[99]->[30]->[40]->tail

    std::cout << "\nSize: " << lst.size() << "\n";

    std::cout << "\nTest find\n";
    std::cout << "Found: " << lst.find(99) << " (expected 99)\n";

    std::cout << "\nTest pop_front\n";
    std::cout << "Popped: " << lst.pop_front() << " (expected 10)\n";
    lst.print();

    std::cout << "\nTest pop_back\n";
    std::cout << "Popped: " << lst.pop_back() << " (expected 40)\n";
    lst.print();

    std::cout << "\nTest pop_middle\n";
    // current list: [20]->[99]->[30]
    std::cout << "Popped at pos 1: " << lst.pop_middle(1) << " (expected 99)\n";
    lst.print(); // expected: h-> [20]->[30]->tail

    std::cout << "\nTest reverse\n";
    lst.push_back(40);
    lst.push_back(50);
    lst.print(); // h-> [20]->[30]->[40]->[50]->tail
    lst.reverse();
    lst.print(); // expected: h-> [60]->[50]->[40]->[30]->tail

    std::cout << "\nTest clear\n";
    lst.clear();
    try{ lst.print(); }
    catch(const std::exception& e){ std::cerr << e.what(); }

    std::cout << "Size: " << lst.size() << "\n";

    return 0;
}

```

Figure 2 shows the main test for the code

The main function then tests all out functions. We didn't test all the invalid statements, because it would be redundant to show all of them.

```
● $ ./main.exe
Test empty list
Empty? yes
Size: 0

Test push_front
head->[10]->[20]->tail

Test push_back
head->[10]->[20]->[30]->[40]->tail

Test push_middle
head->[10]->[20]->[99]->[30]->[40]->tail

Size: 5

Test find
Found: 99 (expected 99)

Test pop_front
Popped: 10 (expected 10)
head->[20]->[99]->[30]->[40]->tail

Test pop_back
Popped: 40 (expected 40)
head->[20]->[99]->[30]->tail

Test pop_middle
Popped at pos 1: 99 (expected 99)
head->[20]->[30]->tail

Test reverse
head->[20]->[30]->[40]->[50]->tail
head->[50]->[40]->[30]->[20]->tail

Test clear
Linkedlist empty
Size: 0
```

Figure 3 shows the main function

As shown in figure 3, the output we get is as expected as the comments in our code.

Exercise 2 - Implementing a stack with an array

```
exercise2 > C stack.h > ...
1  #pragma once
2
3  template <typename Object>
4  class Stack{
5  private:
6      Object* arr;
7      int capacity;
8      int topIndex;
9
10 public:
11     Stack(): arr(new Object[100]), capacity(100), topIndex(-1) {}
12     Stack(int s): arr(new Object[s]), capacity(s), topIndex(-1) {}
13
14     int getCapacity(){
15         return capacity;
16     }
17
18     void push(const Object x){
19         if(topIndex == capacity-1){
20             int newCapacity = capacity*2;
21             Object* NewArray = new Object[newCapacity];
22
23             for(int i = 0; i <= topIndex; i++){
24                 NewArray[i] = arr[i];
25             }
26             delete[] arr;
27             arr = NewArray;
28             capacity = newCapacity;
29         }
30         arr[++topIndex] = x;
31     }
32
33
34     Object pop(){
35         return arr[topIndex--];
36     }
37
38     Object top(){
39         return arr[topIndex];
40     }
41
42     bool empty(){
43         return topIndex == -1;
44     }
45
46     ~Stack(){
47         delete[] arr;
48     }
49
50 };
```

Figure 4 shows the h file for Stack

The main function:

```
exercise2 > main.cpp > main()
1  #include <iostream>
2  #include "stack.h"
3
4  int main() {
5      Stack<int> s(2); // small capacity to trigger resize
6      std::cout << "Capacity of stack: " << s.getCapacity() << "\n";
7      s.push(1);
8      s.push(2);
9      s.push(3); // triggers resize (capacity 2 -> 4)
10     std::cout << "Capacity of stack after resize: " << s.getCapacity() << "\n";
11     s.push(4);
12
13     std::cout << "Pop: " << s.pop() << " (expected 4)\n";
14     std::cout << "Pop: " << s.pop() << " (expected 3)\n";
15     std::cout << "Pop: " << s.pop() << " (expected 2)\n";
16     std::cout << "Pop: " << s.pop() << " (expected 1)\n";
17
18     return 0;
19 }
20
21
```

Figure 5 Shows the main file

The code is fairly short, just showing the push, pop and what happens when the capacity gets filled up to see what happens with resize.

```
Capacity of stack: 2
Capacity of stack after resize: 4
Pop: 4 (expected 4)
Pop: 3 (expected 3)
Pop: 2 (expected 2)
Pop: 1 (expected 1)
```

Figure 6 shows the result in terminal

Exercise 3 – Implementation of Queue

Queue is a template class that implements the interface from queue_class.h.

It is build using 2 Stack members (inStack and outStack)

The idea behind the algorithm is that put(x), always puts Objects in inStack. When you need to get and outstack is empty, we move everything from inStack over to outStack, one at a time, which flips the order so it is the oldest Object first.

That is what makes it into a real FIFO order eventhough it consists of two LIFO stacks.

We only use the four public functions: push, pop, top and empty.

```
template <typename Object>
class Queue{
private:
    Stack<Object> inStack;
    Stack<Object> outStack;
public:
    bool empty();
    Object front();
    Object get();
    void put(const Object x);
};
```

Figure 7 Queue class -queue.h

```
template <typename Object>
Object Queue<Object>::get(){
    if(outStack.empty() == true){
        while(!inStack.empty()){
            outStack.push(inStack.pop());
        }
    }
    return outStack.pop();
}
```

Figure 8 Get function – queue.h

```
int main(){
    Queue<int> q;
    q.put(1);
    q.put(2);
    q.put(3);
    std::cout << q.get() << std::endl; // expected: 1
    std::cout << q.get() << std::endl; // expected: 2
    q.put(4);
    std::cout << q.get() << std::endl; // expected: 3
    std::cout << q.get() << std::endl; // expected: 4
    q.put(5);
    std::cout << q.front() << std::endl; // expected: 5
    std::cout << q.empty() << std::endl; // expected: 0
}
```

Figure 9 test - main.cpp

```
PS C:\Users\au785714\Software Teknologi\Programming\3. Semester\AOS\Hand-ins\Handin2\exercise3> .\main.exe
1
2
3
4
5
0
```

Figure 10 test output

Exercise 4 - Hash function

a) Chaining

Chaining						
Index	Values					Load Factor $n/m=$
0	28					1.43
1	15					
2						
3	17	10				
4						
5	5	19	33	12	33	
6	20					

Figure 11 hash table with chaining

Chaining is taking modulus of the value and assign an index which is equal to the remainder. As an example, with the value of 5. $5 \bmod 11 = 5$. The value 5 is placed in index 5. With chaining, if we get the same index with another value, it will put it at the same index but appended as a node in a linked list. That's also the reason why the load factor can exceed 1. By chaining you are not restricted by the number of indices.

b) Linear probing

Linear Probing		
Index	Values	Load Factor $n/m=$
0	28	1.00
1	15	
2	20	
3	33	
4	12	
5	5	
6	19	

Figure 12 hash table with linear probing

With Linear and Quadratic probing an index can only hold one value. With linear probing if we get the same index, we sum it with N so if we get 5, and the index i occupied: $5+1 = 6$, if place 6 is occupied: $5+2 = 7$, until an empty index is found. The load factor = 1, because we have more value than buckets, when the buckets are filled up we can't store more like with chaining

c)

Quadratic probing		
Index	Values	Load Factor $n/m=$
0	28	1.00
1	15	
2	33	
3	20	
4	17	
5	5	
6	19	

Figure 13 hash table with quadratic probing

Quadratic probing works similarly, but instead jumps forward in increasing squared steps: if index 5 is occupied, it tries $5+1^2 = 6$, then $5+2^2 = 9$ and so on towards N , and so on, which helps spread out collisions more than linear probing does. On index 4, we have 17, and not 12 because the only valid indexes for 12 is: 0, 2, 6, 5 and 6, so the insertion will fail, and we move on to the next value. Same thing applies for the load factor as it did with linear probing

Exercise 5 – Implement a set

Set is a template class, build on our LinkedList class from exercise 1.

We are using the public functions: push_back, pop_front, find and empty from the LinkedList class.

Insert(x) checks with the contains(x) function, and only inserts if the value doesn't exist already, therefore the Set will never have duplicates.

contains(x) uses that the find(x) function from LinkedList throws a "out_of_range" if the value doesn't exist instead of using a bool. Fixed with throw/catch.

LinkedList can only delete at position using pop_middle(pos), and not by value. Fixed in remove(x) by using a temporary list, we put every value in the temporary list, except x, deletes x from original list, and then puts every value from the temporary list back to the original list.

```
template <typename Object>
class Set{
private:
    LinkedList<Object> list;

public:
    void insert(const Object x);
    bool contains(const Object x);
    void remove(const Object x);
};
```

Figure 14 Set class - set.h

```
template <typename Object>
bool Set<Object>::contains(const Object x){
    try {
        list.find(x);
        return true;
    } catch (const std::out_of_range&) {
        return false;
    }
}
```

Figure 15 Contains(x) function - set.h

```

template <typename Object>
void Set<Object>::remove(const Object x){
    LinkedList<Object> temp;

    while(!list.empty()){
        Object val = list.pop_front();
        if(val == x){
            continue;
        }
        else{
            temp.push_back(val);
        }
    }

    while(!temp.empty()){
        Object val = temp.pop_front();
        list.push_back(val);
    }
}

```

Figure 16 remove(x) function - set.h

```

int main(){
    Set<int> s;

    s.insert(1);
    s.insert(2);
    s.insert(3);
    s.insert(2); // dublet: bør ikke ændre noget

    std::cout << "contains(1): " << s.contains(1) << " (expected 1)\n";
    std::cout << "contains(2): " << s.contains(2) << " (expected 1)\n";
    std::cout << "contains(3): " << s.contains(3) << " (expected 1)\n";
    std::cout << "contains(4): " << s.contains(4) << " (expected 0)\n";

    s.remove(2);
    std::cout << "\nAfter remove(2):\n";
    std::cout << "contains(1): " << s.contains(1) << " (expected 1)\n";
    std::cout << "contains(2): " << s.contains(2) << " (expected 0)\n";
    std::cout << "contains(3): " << s.contains(3) << " (expected 1)\n";

    s.remove(99); // Doesn't exist, shouldn't crash or change anything
    std::cout << "\nAfter remove(99) (doesn't exist):\n";
    std::cout << "contains(1): " << s.contains(1) << " (expected 1)\n";
    std::cout << "contains(3): " << s.contains(3) << " (expected 1)\n";
}

```

Figure 17 Test - main.cpp

```
contains(1): 1 (expected 1)
contains(2): 1 (expected 1)
contains(3): 1 (expected 1)
contains(4): 0 (expected 0)

After remove(2):
contains(1): 1 (expected 1)
contains(2): 0 (expected 0)
contains(3): 1 (expected 1)

After remove(99) (doesn't exist):
contains(1): 1 (expected 1)
contains(3): 1 (expected 1)
```

Figur 18 test output

Exercise 6 - implement map using STL vector

```
1  #include <vector>
2  #include <utility>
3  #include <stdexcept>
4  #include <iostream>
5
6
7  template<typename Key, typename Value>
8  class Vector_map{
9      std::vector<std::pair<Key, Value>> vec;
10
11 public:
12     int size(){
13         return vec.size();
14     }
15
16     bool empty(){
17         return vec.size() == 0;
18     }
19
20     void insert(Key key, Value value){
21
22         for(int i = 0; i < vec.size(); i++){
23             if(vec[i].first == key){
24                 vec[i].second = value;
25                 return;
26             }
27         }
28         vec.emplace_back(key, value);
29     }
30
31     Value find(Key key){
32         if(empty()){throw std::out_of_range("Vector empty"); }
33
34         for(int i = 0; i < vec.size(); i++){
35             if(vec[i].first == key){
36                 return vec[i].second;
37             }
38         }
39         throw std::out_of_range("Key not found");
40     }
41 }
42
43
```

Figure 19 shows first part of the h file code

The rest can be seen in the zip file. The functions we use. For testing the code and the implementations:

```
exercised / main.cpp / ...
1  #include "map.h"
2  #include <string>
3
4  int main() {
5      Vector_map<std::string, int> ages;
6
7      std::cout << "Test insert:\n";
8      ages.insert("Alice", 30);
9      ages.insert("Bob", 25);
10     ages.insert("Charlie", 35);
11     ages.print();
12
13     std::cout << "\nTest insert update (same key):\n";
14     ages.insert("Bob", 25); // should update, not duplicate
15     ages.print();
16     std::cout << "Size: " << ages.size() << " (expected 3)\n";
17
18     std::cout << "\nTest find:\n";
19     std::cout << "Alice: " << ages.find("Alice") << " (expected 30)\n";
20     std::cout << "Bob: " << ages.find("Bob") << " (expected 26)\n";
21
22     std::cout << "\nTest contains:\n";
23     std::cout << "Contains Charlie? " << (ages.contains("Charlie") ? "yes" : "no") << "\n";
24     std::cout << "Contains David? " << (ages.contains("David") ? "yes" : "no") << "\n";
25
26     std::cout << "\nTest remove:\n";
27     ages.remove("Alice");
28     ages.print();
29     std::cout << "Size: " << ages.size() << " (expected 2)\n";
30
31     std::cout << "\nTest with different types: int & string:\n";
32     Vector_map<int, std::string> idToName;
33     idToName.insert(1, "one");
34     idToName.insert(2, "two");
35     idToName.insert(3, "three");
36     idToName.print();
37
38     return 0;
39 }
```

Figure 20 shows the main function for the map

Output:

```
=== Test insert ===  
[1] Key: Alice, Value: 30  
[2] Key: Bob, Value: 25  
[3] Key: Charlie, Value: 35  
  
=== Test insert update (same key) ===  
[1] Key: Alice, Value: 30  
[2] Key: Bob, Value: 26  
[3] Key: Charlie, Value: 35  
Size: 3 (expected 3)  
  
=== Test find ===  
Alice: 30 (expected 30)  
Bob: 26 (expected 26)  
  
=== Test contains ===  
Contains Charlie? yes  
Contains David? no  
  
=== Test remove ===  
[1] Key: Bob, Value: 26  
[2] Key: Charlie, Value: 35  
Size: 2 (expected 2)  
  
=== Test with different types: int -> string ===  
[1] Key: 1, Value: one  
[2] Key: 2, Value: two  
[3] Key: 3, Value: three
```

Figure 21 shows the output of the terminal

The functions work as intended.

Exercise 7 - Quadratic probing

Quadratic probing		
Index	Values	Load Factor $n/m=$
0	22	0.73
1	22	
2	202507614	
3		
4	22	
5	5	
6	16	
7		
8	202411378	
9	27	
10		

Figure 22 Shows quadratic probing

For this task we did Quadratic probing again. We used the values given from the task, which was placed correctly, then we used our ages: 22 & 22, and our student ID: 202507614 and then 202411738, which should give us the indices shown in Figure .